

Evaluación del desempeño de modelos MLP en la clasificación de dígitos y letras manuscritas

Xavier Muñoz Díaz

Resumen: Este informe evalúa la precisión de 3 MLP para clasificar dígitos y letras manuscritas utilizando los datasets EMNIST_Digits y EMNIST_Letters. Cada modelo en cada experimento fue entrenado y evaluado 5 veces y se obtuvieron métricas como el accuracy y el tiempo de ejecución. El primer conjunto de datos presenta una mayor precisión al clasificar dígitos (99.08%) en comparación con el segundo al hacerlo con letras (90.19%), lo cual se debe a las características que posee dicho dataset.

Palabras clave: MLP, EMNIST, clasificación de imágenes, redes neuronales.

1. Introducción

El reconocimiento de caracteres manuscritos es una tarea importante debido a que es posible aplicarlo a diversas herramientas. Por ejemplo, aplicaciones de lectura automática de documentos y la validación de cheques bancarios. En este contexto, se pueden utilizar los modelos de perceptrones multicapa (MLP) para abordar este tipo de problemas que requieren clasificación. Sin embargo, el rendimiento de estos depende en gran medida del diseño, como el número de capas ocultas, las funciones de activación, las funciones de pérdida y las estrategias de inicialización de pesos.

El conjunto de datos EMNIST proporciona subconjuntos de imágenes que incluye tanto dígitos como letras manuscritas, con la finalidad de evaluar la capacidad de los modelos creados para clasificar información visual en múltiples clases.

Para este trabajo, se utilizarán dos subconjuntos del dataset: EMNIST_Digits, que contiene 10 clases correspondientes a dígitos del 0 al 9, y EMNIST_Letters, que incluye 26 clases representando letras del abecedario inglés, cuyo objetivo consiste en entender y aplicar redes neuronales para reconocimiento de dígitos y letras manuscritas utilizando los subconjuntos del dataset mencionado.

El informe presenta cómo se desarrolló el experimento y las configuraciones que se utilizan para el entrenamiento de los modelos. Luego se analizan los resultados entregados y finalmente se presenta la conclusión del trabajo considerando los resultados obtenidos.

2. Desarrollo

Para poder reconocer dígitos y letras, se implementaron 3 MLP para cada experimento, utilizando un entorno de programación en Python 3.11.4.

El primer modelo utiliza el subconjunto de datos EMNIST_Digits que posee 240.000 imágenes para entrenamiento y 40.000 para prueba, mientras que el segundo utiliza 124.800 y 20.800 respectivamente.

En primera instancia, dichos conjuntos se cargan y en el procesamiento cada imagen se convierte a un vector de 1D y se normaliza con valores entre 0 y 1 al dividir en 255 para mejorar el rendimiento de los modelos.

Previo a ejecutar los experimentos con hiperparámetros específicos, se crearon, entrenaron y evaluaron diversos MLP con distintas capas ocultas, funciones de activación y de pérdida, con la finalidad de obtener la mejor configuración para cada modelo utilizando como métrica el accuracy total. Además, se utilizaron optimizadores como Adam que utiliza una media móvil exponencial de los gradientes para ajustar las tasas de aprendizaje; NaDam para acelerar el descenso de la curva de gradiente (Gavilán, 2020); y AdamW que es un método de optimización estocástica que modifica la implementación de Adam (Papers With Code - AdamW Explained, s. f.).

Para el experimento 1 y 2 se utilizan los siguientes modelos:

- MLP1: 2 capas ocultas de 256 y 128 neuronas, función de activación ReLU y función de pérdida `categorical_crossentropy`.

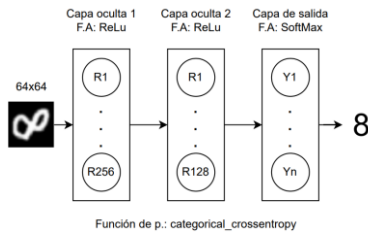


Figura 1: Diagrama de bloques de MLP1.

- MLP2: 3 capas ocultas de 256, 128 y 64 neuronas, función de activación Swish y función de pérdida sparse_categorical_crossentropy.

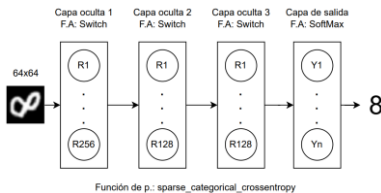


Figura 2: Diagrama de bloques de MLP2.

- MLP3: 4 capas ocultas de 512, 256, 128 y 64 neuronas, función de activación Leaky ReLU y función de pérdida categorical_crossentropy.

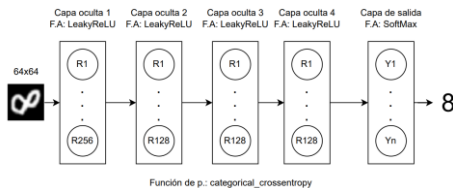


Figura 3: Diagrama de bloques de MLP3.

El código incluye la visualización de 20 imágenes para el conjunto de entrenamiento y 20 para el de prueba de forma aleatoria, y estas son etiquetadas con el archivo mapping que incluye el subconjunto. Este proceso se realiza para el experimento 1 y 2.

3. Resultados experimentales y discusión

Al observar el gráfico de barras de la Figura 4, se observa que en general el accuracy por clase es similar en los modelos y sin mucha variación debido a que en sí el dataset EMNIST_Digits con los modelos posee un accuracy promedio de 99,08%, sin embargo, las diagonales de las matrices de confusión de las Figuras 5, 6 y 7 nos entregan mayor información, ya que indica que los valores más bajos son en las clases: 2, 3, 8 y 9 para el MLP1; 5, 7, 8 y 9, para el MLP2; y 2, 3, 4, 8 y 9 para el MLP3. De esto se puede extraer

que las clases más complejas de clasificar son 2, 3, 8 y 9, lo cual puede deberse a las similitudes en patrones que puedan tener estas clases dificultando su discriminación. Sin embargo, el modelo que mejor clasifica las clases 2, 8 y 9 es el MLP2, pero el MLP1 posee una clasificación más precisa en general. Esto se observa en la Tabla 1 que contiene los accuracy totales por clase y se desprende que el MLP1 es el que posee mejor valor con un 99.09%, seguido por el MLP2 (99.08%) y por el MLP3 (99.01%) y el tiempo de ejecución de menor a mayor se presenta en el mismo orden, y se da cuenta que el MLP3 se demora casi el doble que el MLP1. En consecuencia, es posible afirmar que un modelo más complejo no implica mejores resultados. Por otro lado, las clases que mejor se clasifican son la 0 y la 1.

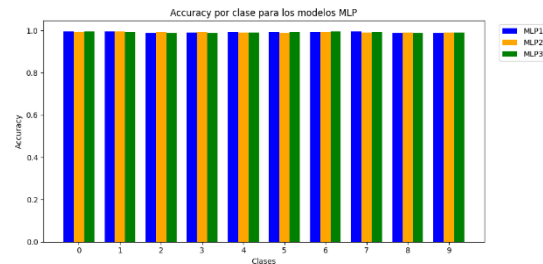


Figura 4: Gráfico de barra que muestran el accuracy por clase resultante del experimento 1 para cada modelo.

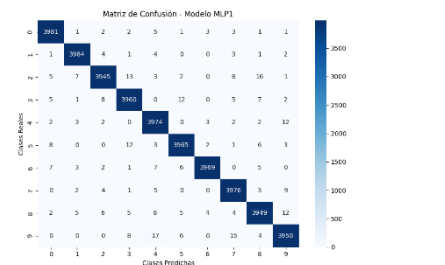


Figura 5: Matriz de confusión resultante del MLP1 para el experimento 1.

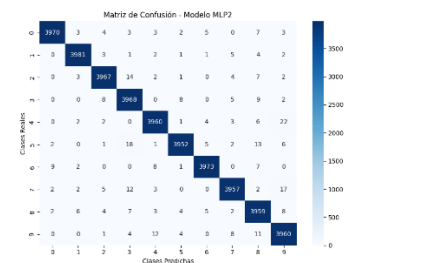


Figura 6: Matriz de confusión resultante del MLP2 para el experimento 1.

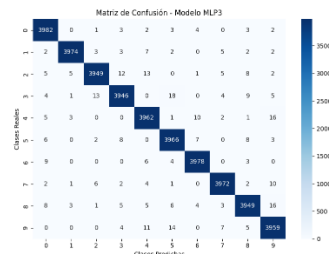


Figura 7: Matriz de confusión resultante del MLP3 para el experimento 1.

Modelo	MLP1	MLP2	MLP3
Mediana 5 iteraciones	0,9909	0,9908	0,9901
Tiempo ejecución[s]	1.670,097	1.859,389	3.104,886
Accuracy total	0,991325	0,991175	0,990925

Tabla 1: Métricas obtenidas para el experimento 1.

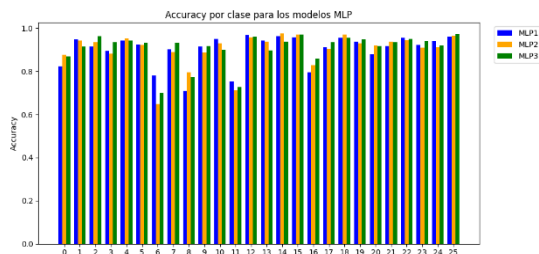


Figura 8: Gráfico de barra que muestra el accuracy por clase resultante del experimento 2 para cada modelo.

Modelo	MLP1	MLP2	MLP3
Mediana 5 iteraciones	0,9019	0,9028	0,9015
Tiempo ejecución[s]	1.423,652	1.064,052	1.995,123
Accuracy total	0,902307	0,9011057	0,907355

Tabla 2: Métricas obtenidas para el experimento 2.

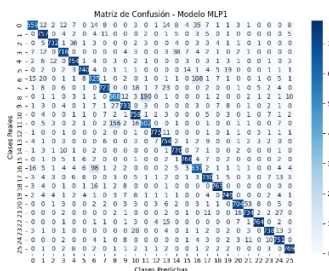


Figura 9: Matriz de confusión resultante del MLP1 para el experimento 2.

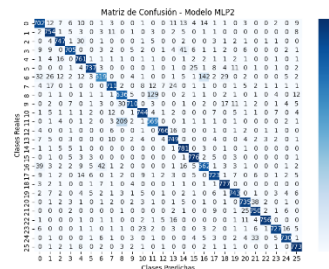


Figura 10: Matriz de confusión resultante del MLP2 para el experimento 2.

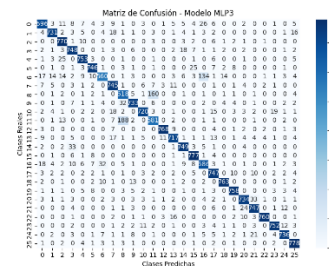


Figura 11: Matriz de confusión resultante del MLP3 para el experimento 2.

A diferencia del experimento 1, al observar el gráfico de barras de la Figura 8 se aprecia una clara variación en el accuracy por clase entre los modelos cuyas clases 6, 8 y 11 son las más complejas de clasificar.

Las diagonales de las matrices de confusión de las Figuras 9, 10 y 11 muestran que a los modelos les acompleja clasificar las clases: 0, 6, 8, 11 y 16 para el MLP1; 6, 8, 11 y 16, para el MLP2; y 6, 8, 11 y 16 para el MLP3. Las clases en cuestión corresponden a las letras: "A/a" para la clase 0; "G/g" para la clase 6; "I/i" para la clase 8; "L/l" para la clase 11; y "Q/q" para la clase 16. Esto es esperable debido a la similitud que presentan las clases 0, 6 y 16, y 8 con 11. Sin embargo, el MLP2 clasifica peor la clase 6 y 11 pero mejor la 8 y la 16 a comparación del MLP1, mientras que el MLP3 presenta más dificultades con las clases 6 y 11 versus el MLP1, y con la clase 8 a comparación del MLP2, pero clasifica mucho mejor la clase 16 en comparación de los modelos anteriores. Además, el MLP1 clasifica mejor las clases 12, 14 y 25 que corresponden a las letras "M/m", "O/o" y "Z/z" respectivamente, el MLP2 lo hace para las clases 14, 15 y 18 cuyas letras son "O/o", "P/p" y la "S/s" en comparación del MLP1, mientras que el MLP3 posee mejor precisión a

comparación de los otros 2 modelos con las clases 15 y 25.

Sin embargo, a pesar de las falencias del MLP3 al clasificar, es el modelo que presenta mejor accuracy total (90,73%) pero posee el mayor tiempo de ejecución como se muestra en la Tabla 2, debido a las características de configuración del modelo como se menciona en la sección anterior. Mientras que el MLP2 tiene el menor tiempo de ejecución junto con el accuracy total, pero con la mejor mediana.

Al comparar ambos experimentos se observa que el dataset EMNIST_Letters presenta mayor complejidad de clasificación que EMNIST_Digits. De hecho, el accuracy promedio del dataset es de 90,19%, casi 9% de diferencia respecto al conjunto de datos de dígitos. Esto puede ser atribuido a la mayor variabilidad y similitud visual entre las clases de letras manuscritas, en comparación con las clases de dígitos. Por lo mismo, los modelos del subconjunto de dígitos presentan un desempeño similar versus el subconjunto de letras, cuyo desempeño varía bastante dependiendo del modelo y de la clase. Además, los tiempos de ejecución son mayores en el experimento 1 debido a la gran cantidad de imágenes que posee el dataset.

4. Conclusiones

Las ventajas de los MLP es que poseen una alta adaptabilidad a diversos tipos de datos mediante la configuración de hiperparámetros y capas, pero se debe tener en cuenta que el tiempo de ejecución puede ser más alto como se observó en el MLP3 de ambos experimentos. Además, los resultados obtenidos permiten concluir que los modelos MLP son efectivos para la clasificación de caracteres manuscritos, alcanzando un alto nivel de precisión en el dataset EMNIST_Digits (99.08%). Sin embargo, en EMNIST_Letters, debido a la complejidad del dataset al poseer mayor cantidad de clases y la similitud visual que existe entre estas, su desempeño es menor (90.19%). En consecuencia, es posible afirmar que es importante diseñar modelos considerando las características del conjunto de datos con el que se trabaja.

Para trabajos futuros, se podría implementar otros optimizadores en los modelos, y también diseñar modelos más robustos que sean específicos para EMNIST_Letters dada su

complejidad y comparar su precisión al utilizar los mismos modelos en EMNIST_Digits. También sería interesante experimentar con CNN debido a su experticia en tareas de clasificación y visión por computadora (IBM, 2024).

Finalmente, este laboratorio me permitió relacionar los conceptos teóricos aprendidos en la cátedra con su aplicación práctica. Además, fue posible comprobar cómo el diseño del modelo afecta directamente el desempeño.

5. Referencias

- IBM. (2024, 21 agosto). ¿Qué son las redes neuronales convolucionales? <https://www.ibm.com/es-es/topics/convolutional-neural-networks>
- Jaiswal, S. (2024, 11 septiembre). Perceptrones multicapa en el aprendizaje automático: Guía completa. Datacamp. <https://www.datacamp.com/es/tutorial/multilayer-perceptrons-in-machine-learning>
- Gavilán, I. G. (2020, 27 mayo). Catálogo de componentes de redes neuronales (y IV): optimizadores. Ignacio G.R. Gavilán. <https://ignaciogavilan.com/catalogo-de-componentes-de-redes-neuronales-y-iv-optimizadores/>
- Papers with Code - AdamW Explained. (s. f.). <https://paperswithcode.com/method/adamw>